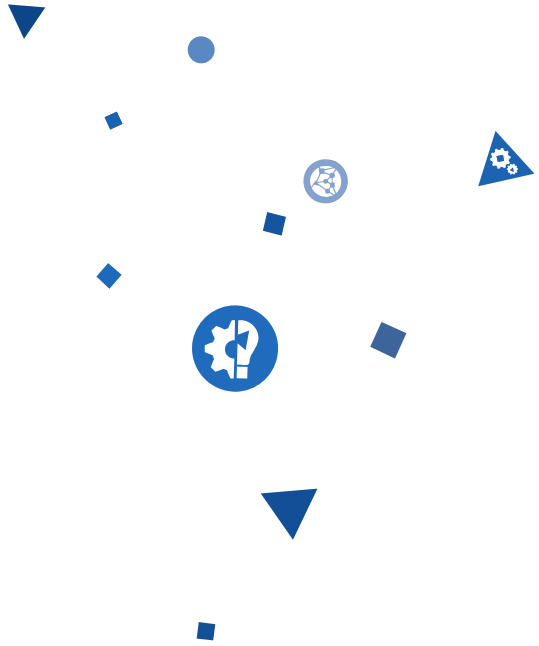


From Bytecode to Native Code in HotSpot JVM

Ionuţ Baloşin

Software Architect
ibalosin@luxoft.com
@ibalosin

About Me

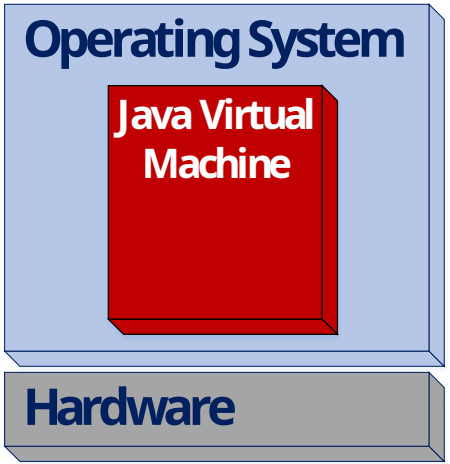
- **Software Architect** @ www.luxoft.com
  - 9 years of software development experience
- **Technical Trainer** @ www.luxoft-training.com
  - "*Java Performance and Tuning*" course
  - "*Software Architecture Methodology*" course

Agenda

- Understanding Bytecode
- How to Disassemble Bytecode
- Bytecode Execution Workflow
  - Interpreter
  - JIT Compiler
- How to see Native Code from Bytecode
- Demo
- Micro-benchmarking advice

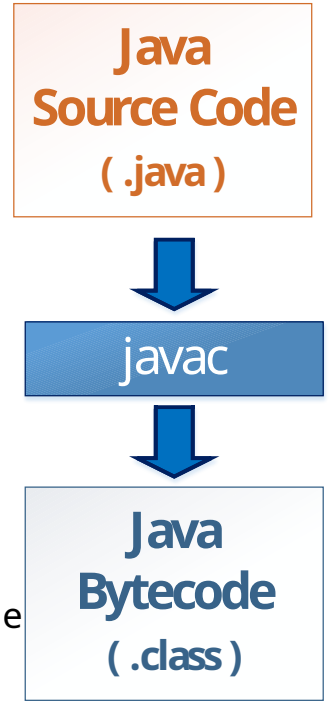
HotSpot Java Virtual Machine

- JVM
  - process virtual machine for application code
  - built-in memory management system
  - provides robust and secure execution
  - stack-based implementation



javac

- Bytecode is generated using **javac** based on Java source code
- There are about 200 bytecode instructions (**opcodes**) in use
  - each **opcode** is represented by a single byte → **bytecode**
  - **opcodes** are organized in families
- Bytecode does not have direct access to any memory location
- Bytecode does not contain some high-level features present in the Java Source Code
  - E.g. comments, for, while, etc

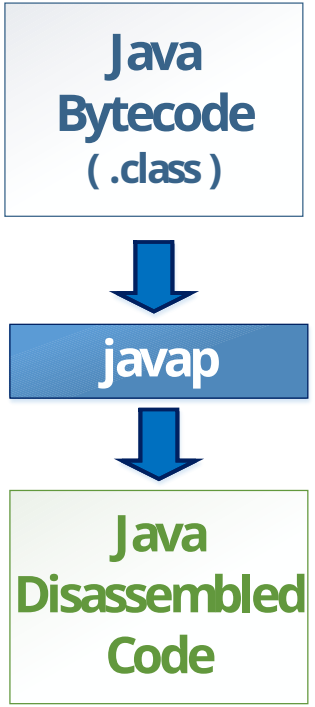


javap

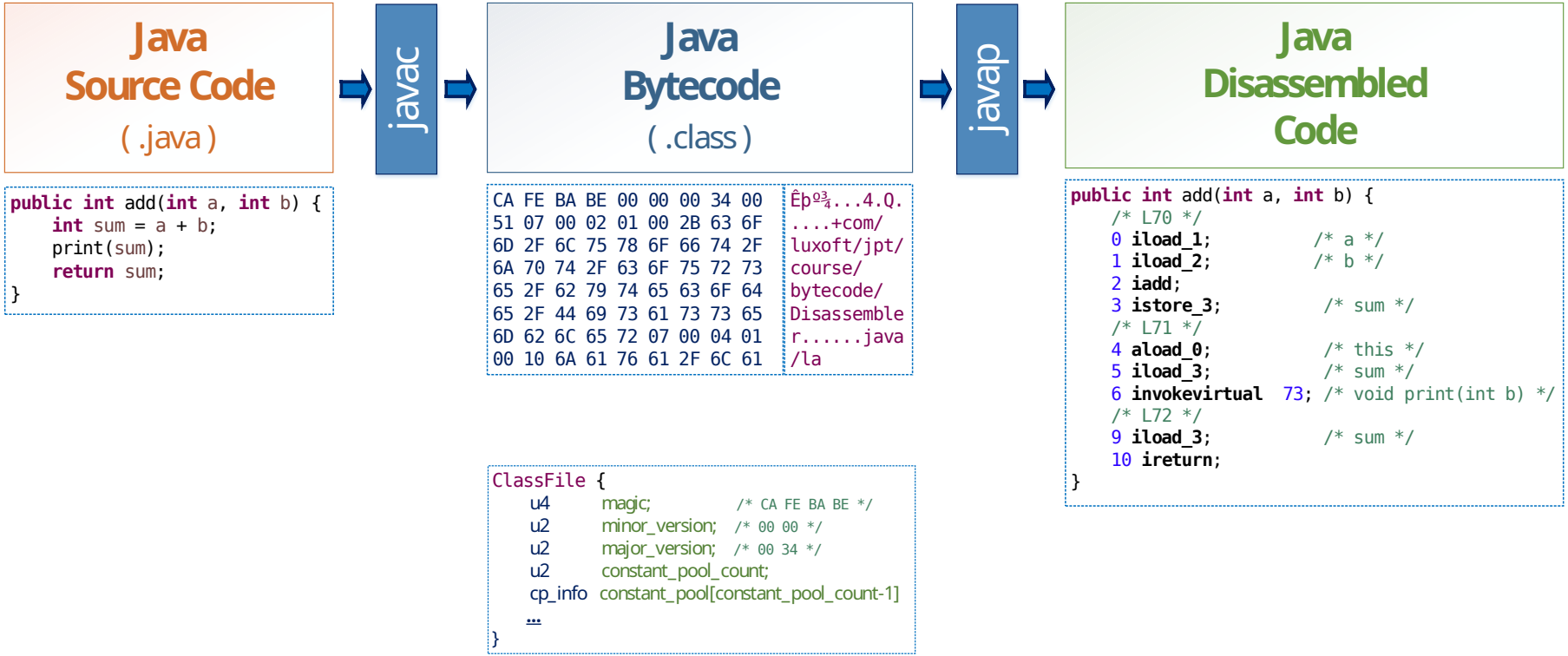
- Disassembled code is generated using **javap** based on generated Bytecode

- **javap** - Java Class File Disassembler
  - by default it shows public and default methods
  - javap -c <path_to_class>

- IDE Disassemblers:
  - Eclipse: Bytecode Visualizer
  - IntelliJ IDEA: ASM Bytecode Outline



From Source Code to Disassembled Code



OpCode Families

```
public int add(int a, int b) {
    /* L70 */
    0 iload_1;                      /* a */
    1 iload_2;                      /* b */
    2 iadd;
    3 istore_3;                     /* sum */
    /* L71 */
    4 aload_0;                      /* this */
    5 iload_3;                      /* sum */
    6 invokevirtual  73;            /* void print(int b) */
    /* L72 */
    9 iload_3;                      /* sum */
    10 ireturn;
}
```

ARITHMETIC OPCODES	
add	add two values from top of stack and store the result
sub	subtracts two values from top of stack and store the result
div	divide two values from top of stack and store the result
mult	multiplies two values from top of stack and store the result

INVOCATION OPCODES	
invokeinterface	calls an interface method
invokespecial	calls a non-overridable instance method
invokevirtual	calls a regular overridable instance method
invokestatic	calls a static method
invokedynamic	dynamic invocation

LOAD & STORE OPCODES	
load	load a value from Local Variable Array into the stack
ldc	load a constant from the Constant Pool into the stack
store	store a value into Local Variable Array and remove it from stack
dup	duplicate the value on top of the stack

Local Variable Array

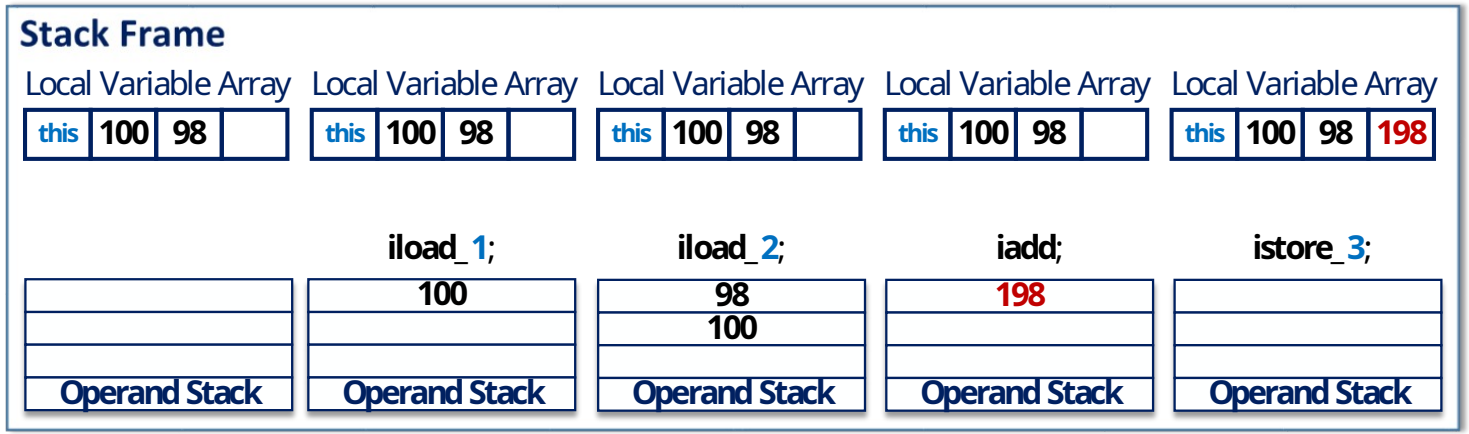
```
public int add(int a, int b) {
    int sum = a + b;
    print(sum);
    return sum;
}
```

Local Variable Array
index

```
public int add(int a, int b) {
    /* L70 */
    0 iload_1;                      /* a */
    1 iload_2;                      /* b */
    2 iadd;
    3 istore_3;                     /* sum */
    /* L71 */
    4 aload_0;                      /* this */
    5 iload_3;                      /* sum */
    6 invokevirtual  73;            /* void print(int sum) */
    /* L72 */
    9 iload_3;                      /* sum */
    10 ireturn;
}
```

How Operand Stack and Local Variable Array work

```
public int add(int a, int b) {
    /* ... */
    0 iload_1;                  /* a */
    1 iload_2;                  /* b */
    2 iadd;
    3 istore_3;                 /* sum */
    /* ... */
}
```



Runtime Constant Pool

```
public int add(int a, int b) {
    int sum = a + b;
    print(sum);
    return sum;
}
```

```
public int add(int a, int b) {
    /* L70 */
    0 iload_1;                  /* a */
    1 iload_2;                  /* b */
    2 iadd;
    3 istore_3;                 /* sum */
    /* L71 */
    4 aload_0;                  /* this */
    5 iload_3;                  /* sum */
    6 invokevirtual  73;        /* void print(int sum) */
    /* L72 */
    9 iload_3;                  /* sum */
    10 ireturn;
}
```

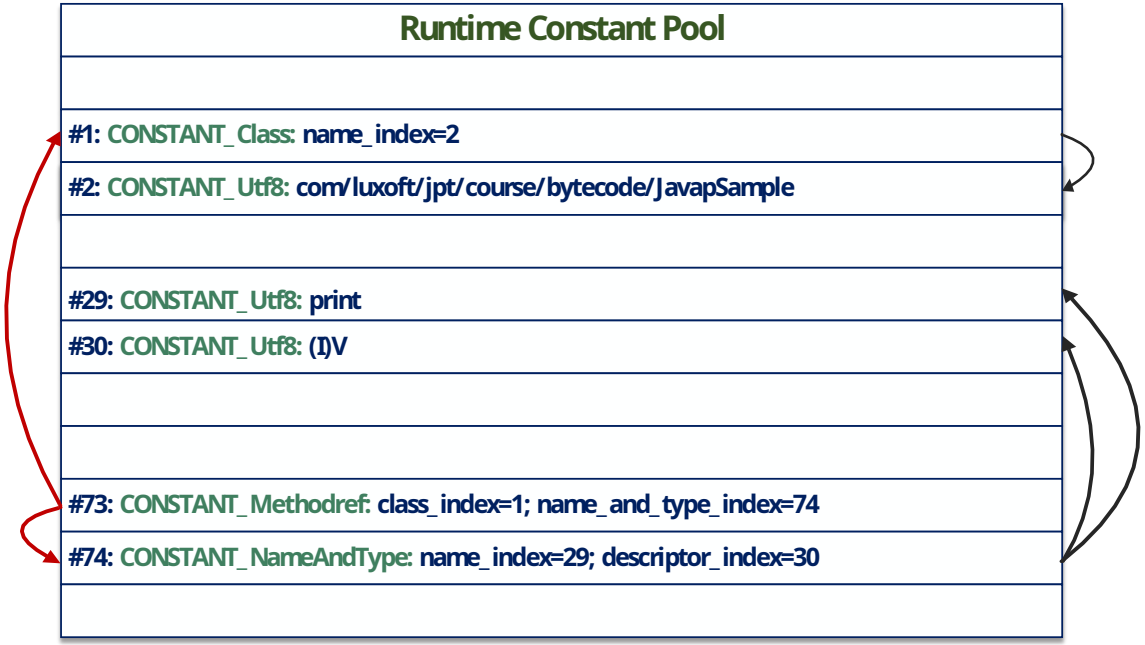
Local Variable Array
index

Runtime Constant Pool
reference

How Runtime Constant Pool works

- Ordered set of constants used by the type
- Entries have usually 2 bytes, referenced by index
- Plays a central role in the dynamic linking of Java

```
public int add(int a, int b) {
    /* ... */
    6 invokevirtual 73;       /* void print(int sum) */
    /* ... */
}
```



Exceptions handling at Bytecode level

```java
public static int remainder(int a, int b) throws DivideByZeroException {
    try {
        return a % b;
    } catch (ArithmeticException e) {
        throw new DivideByZeroException();
    }
}
```

```
public static int remainder(int a, int b) throws com.luxoft.jpt.course.bytecode.DivideByZeroException {
    /* L26 */
    0 iload_0;                /* a */
    1 iload_1;                /* b */
    2 irem;
    3 ireturn;
    /* L27 */
    4 astore_2;               /* e */
    /* L28 */
    5 new 17;
    8 dup;
    9 invokespecial 19;       /* com.luxoft.jpt.course.bytecode.DivideByZeroException() */
    12 athrow;

    /*                          ExceptionTable                          */
    /* ----------+--------+------------+------------------------------ */
    /*  start_pc | end_pc | handler_pc |                    catch_type  */
    /* ----------+--------+------------+------------------------------ */
    /*         0 |      3 |          4 | java.lang.ArithmeticException  */
    /* ----------+--------+------------+------------------------------ */

}
```

Quizz

- Spot few ambiguities based on below decompiled class:

```
public class MyDouble implements MyDoubleInterface<Integer> {

    @Override
     public Integer getDouble() {
         return Integer.valueOf(10) * 2;
     }
}
```

```
public class MyDouble implements com.luxoft.jpt.course.bytecode.MyDoubleInterface<java.lang.Integer> {

    /* compiled from MyDouble.java */

    public MyDouble() {...}

    public java.lang.Integer getDouble() {...}

    public volatile java.lang.Object getDouble() {...}
}
```

Quizz

- Where method **access$0** comes from?

```
public class JavapOuterClass {

    private void foo() {
    }

    public class MyInnerClass {
        public void run() {
            foo();
        }

    }
}
```

```
public class JavapOuterClass {

    /* compiled from JavapOuterClass.class */

    public JavapOuterClass() {...}

    private void foo() {...}

    static void access$0(com.luxoft.jpt.course.bytecode.JavapOuterClass arg0) {
        arg0.foo();
    }
}
```

Quizz

- Where method **$0** comes from? How it could be **private** and **static** inside the Interface?

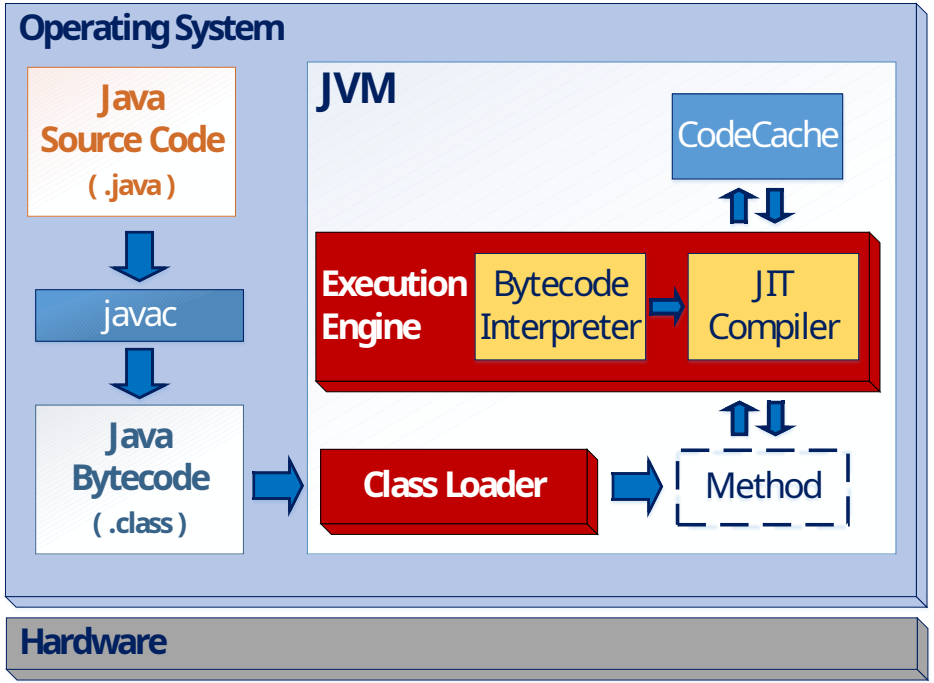
```
public interface MyPrinter {

    default void print() {
        Runnable r = () -> { System.out.println("Hello Voxxed Bucharest"); };
        r.run();
    }
}
```

```
public interface MyPrinter {

    /* compiled from MyPrinter.class */
    default void print() {
        invokedynamic [ LambdaMetafactory.metafactory,
                        MethodHandle(Runnable.run),
                        MethodHandle(lambda$0) ] ()
    }

    private static void lambda$0() {
       invokevirtual java.lang.System.out.println("Hello Voxxed Bucharest")
    }
```

Execution Engine

- **HotSpot** rationales
  - Weak Generation Hypothesis
    - **memory management and Garbage Collector**
  - use runtime to take best optimization decisions
    - **Execution Engine**

- **Execution Engine**
  - Interpreter
  - Just-In-Time Compiler



HotSpot Interpreter

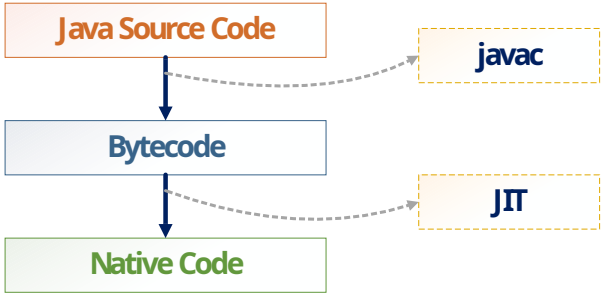
- When the JVM first starts up, thousands of methods are called
  - compiling all methods can significantly affect startup time
  - "*~90% of time is spent in ~2% of methods*"
- Solution: methods are not compiled first time they are called, they are **interpreted**
  - HotSpot JVM starts running in Interpreter mode

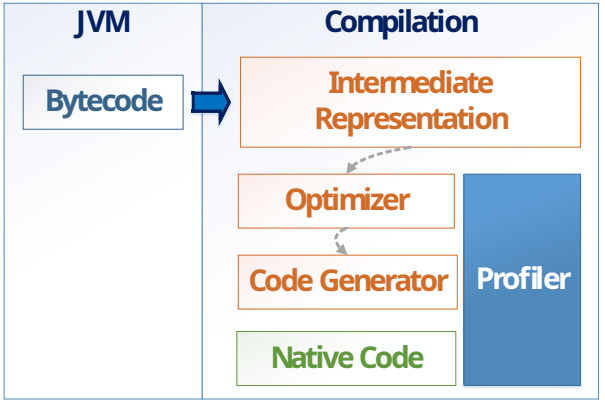
- Each method have a call count, once it reaches an **invocation threshold** it gets compiled
  - hence **hot spots** (HotSpot JVM)

JVM argument	Explanations
-XX:CompileThreshold	Number of method invocations/branches before compiling [client 1.5k invocations; server 10k invocations]

HotSpot Just-In-Time Compiler

- **Client** (-client) **C1**
  - fast code generation of acceptable quality
  - rock-solid and proved optimizations
  - doesn't need a **Profiler**
  - compilation threshold: **1,5k** invocations
- **Server** (-server) **C2**
  - highly optimized code
  - many aggressive and speculative optimizations
  - does need a **Profiler**
  - compilation threshold: **10k** invocations
- **Tiered Mode** (-XX:+TieredCompilation) **C1 + C2**
  - Level 0 = Interpreter
  - Level 1-3 = **C1**
    - **C1** without profiling
    - **C1** with basic profiling
    - **C1** with full profiling
  - Level 4 = **C2**





Welcome to HotSpot Optimizations' World

Just-In-Time Compiler optimizations

- Dynamic de-virtualization
  - if JIT only sees 1 implementation of a virtualized method
    - it becomes a mono-morphic call
- Dynamic de-optimization
  - if JIT discovers another implementation it must *de-optimize* and might *re-optimize* for second implementation
    - it becomes a bi-morphic call
- Intrinsics
  - optimizations done specifically to CPU
  - do not inline, but inserts “best” native code
  - Examples: CAS, String::equals; Math::*; System::arrayCopy; Object::hashCode

Method inlining

```
double getInterestRate() {
    return (canComputeInterestRate()) ? 0 : (3 * amount / 1200);
}

boolean canComputeInterestRate() {
    return amount != 0;
}
```

```
double getInterestRate_() {
    return (amount != 0) ? 0 : (3 * amount / 1200);
}
```

HotSpot JVM arguments

JVM argument	Explanations
-Djava.compiler=none	Disables JIT Compiler
-client	Activates client (C1) compilation
-server	Activates server (C2) compilation
-XX:+TieredCompilation	Activates tiered (C1+C2) compilation
-Xbatch	Switches from background to foreground JIT compilation task until completed. It gets more deterministic behavior
-XX:+PrintCompilation	Prints time spent in JIT Compiler
-XX:+PrintInlining	Prints what methods get inlined
-XX:-CITime	Prints time spent in JIT Compiler.
-XX:MaxInlineSize	Specifies maximum number of bytecode instructions in a method which gets inlined. It does not to look at the invocation count [default 35bytes]

How to see Native Code from Bytecode

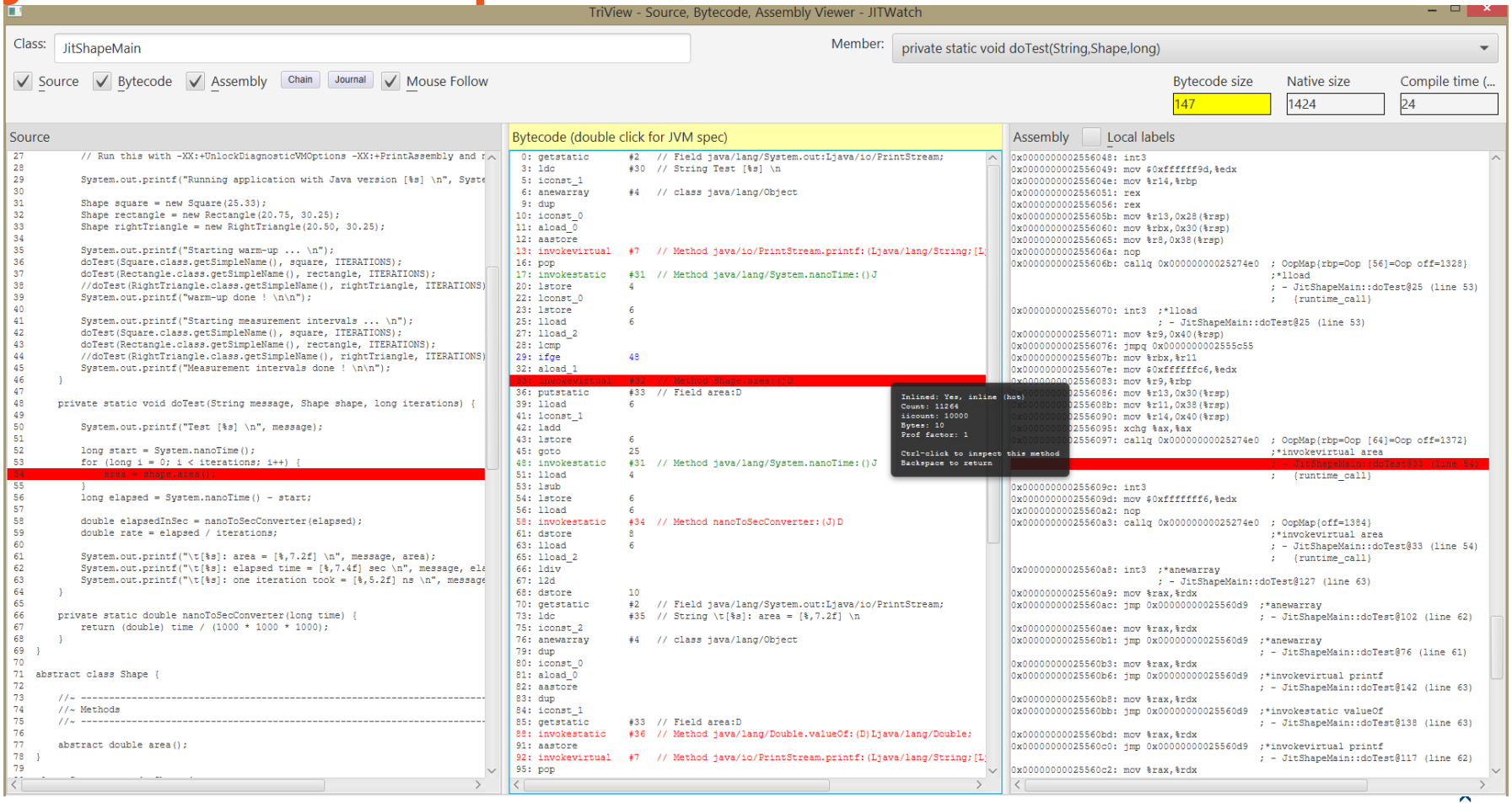
- **HSDIS**
  - https://wiki.openjdk.java.net/display/HotSpot/PrintAssembly

JVM argument	Description
-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly	Activates for assembly printing

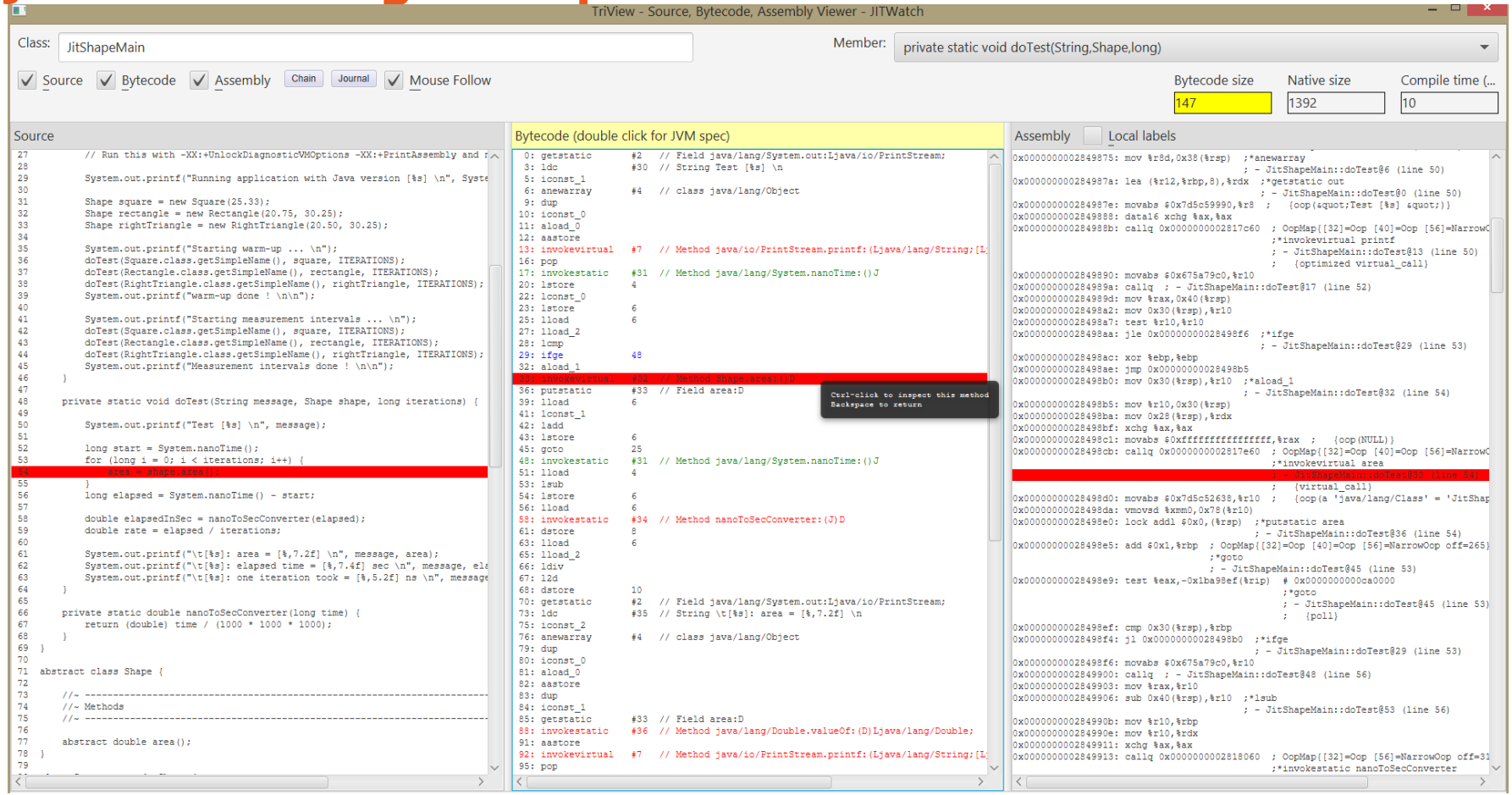
Demo Time

- Direct vs. Getter/Setter fields access members
  - interpreted mode
  - mixed compiled mode
- Mono-morphic / bi-morphic / mega-morphic virtual method calls
- JIT Watch

JITWatch - bi-morphic call



JITWatch - mega-morphic call



Micro-Benchmarking advice

- **Avoid measuring in Interpreter while benchmarking**
- **Have appropriate # of warmup iterations to reach steady state**
  - Ideally several cycles of short warm ups
- **Be aware of Just-In-Time Compiler optimizations**
  - Virtual method calls optimizations
  - Inlining
  - On-Stack-Replacement

THANK YOU

Ionuţ Baloşin

Software Architect
ibalosin@luxoft.com
@ibalosin

LUXOFT

LXFT LISTED NYSE

Appendix

LUXOFT

LXFT
LISTED
NYSE

PrintCompilation

- 202 15 % 3 ! s b n com.luxoft.jpt.course.jit.fieldaccess.DfaCaller::call @ 16 (51 bytes)
  - 202 : millis from start JVM
  - 15: sequence number of compilation
  - %: On Stack Replacement
  - 3: Compilation Level
  - !: method has exception handlers
  - s: synchronized method
  - b: compilation occurred in blocking mode
  - n: compilation occurred for a wrapper to a native method
  - 16: On Stack Replacement bytecode index {osr_bci}
  - 51 bytes: number of bytes contained in compiled method

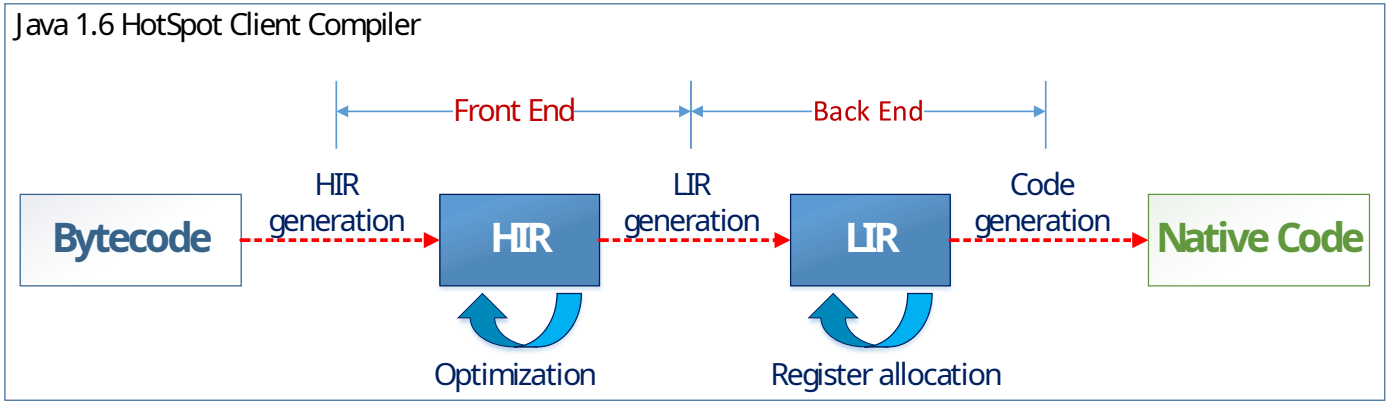
Code Cache

- Code Cache – holds JVM compiled code (set of assembly-language instructions)
  “hidden space” cleaned by JIT Compiler threads

- **Warnings:**
  - CodeCache is full. Compiler has been disabled
  - CodeCache is full. Try increasing the code cache size using -XX:ReservedCodeCacheSize

JVM arguments	Explanations
-XX:InitialCodeCacheSize	Initial code cache size (in bytes)
-XX:ReservedCodeCacheSize	Maximum/reserved code cache size to be used
-XX:+UseCodeCacheFlushing	Attempt to sweep the code cache before shutting off compiler. Default with Java 7u4

Front End / Back End Compiler



- **SSA –** Static Single Assignment
- **CFG** – Control Flow Graph
- **HIR –** High Level Intermediate Representation
- **LIR –** Low Level Intermediate Representation

- **HIR** optimizations:
  - method inlining
  - loop unrolling
  - null check elimination

- **LIR** register allocation:
  - maps virtual to physical registers

Just-In-Time Compiler optimizations

Loop unrolling – reduces cost of jump instructions that control the loop

```
for (x = 0; x < 100; x++) {
    compute(x);
}
```

```
for (x = 0; x < 100; x += 5) {
    compute(x);
    compute(x + 1);
    compute(x + 2);
    compute(x + 3);
    compute(x + 4);
}
```

Null check elimination – eliminates unreachable null code branch

```
private void enroll() {
    JavaPerformanceAndTuning jtp = new JavaPerformanceAndTuning();
    if (null == jtp)
        throw new NullPointerException("Cannot enroll to null course");
    jtp.enroll();
}
```

Just-In-Time Compiler optimizations

Lock Eliding – when only the thread that created the lock will ever have access to it, can safely be omitted

```
public void foo() {

    List list = new ArrayList();
    synchronized (list) {
        list.add(getElement());
    }

}
```

```
public void foo() {

    List list = new ArrayList();
    list.add(getElement());

}
```

Lock coarsening – merges two critical sections, along with the non-critical section, into a larger block

```
synchronized (object) {
   /* critical section */
}
```

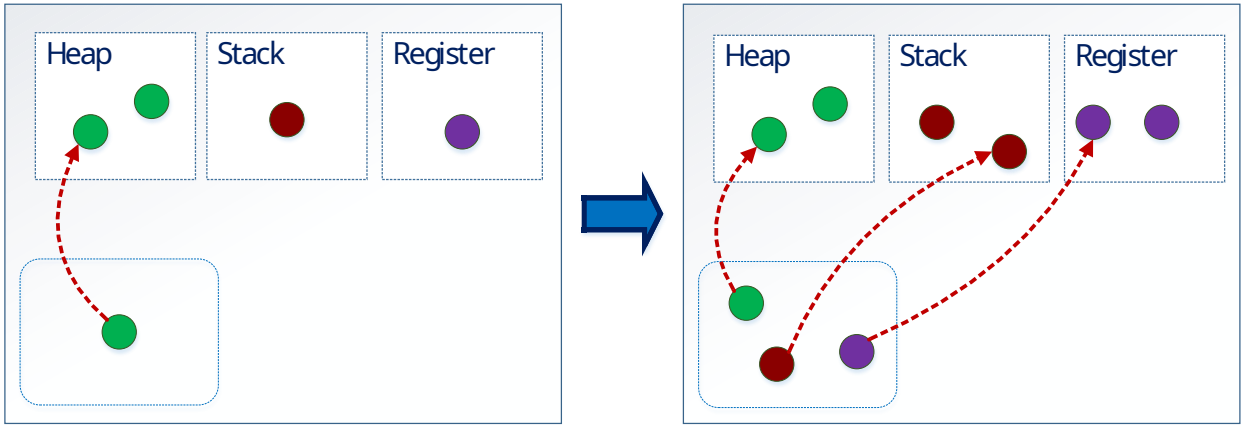
```
{
  /* non-critical section */
}
```

```
synchronized (object) {
   /* critical section */
}
```

```
synchronized (object) {
   /* critical section */
   /* non-critical section */
   /* critical section */
}
```

Just-In-Time Compiler optimizations

- Escape Analysis - analyze the scope of a new object's and decide whether to allocate it on the Java heap
  - uses **scalar replacement** technique which removes object allocation by “object explosion” (prefer Register allocation, spills to Stack if necessary)



JVM arguments	Explanations
-XX:+DoEscapeAnalysis	Allocation optimization. Enabled by default in Java 6 u23